

LECTURE – ASSOCIATION OOP (JAVA)

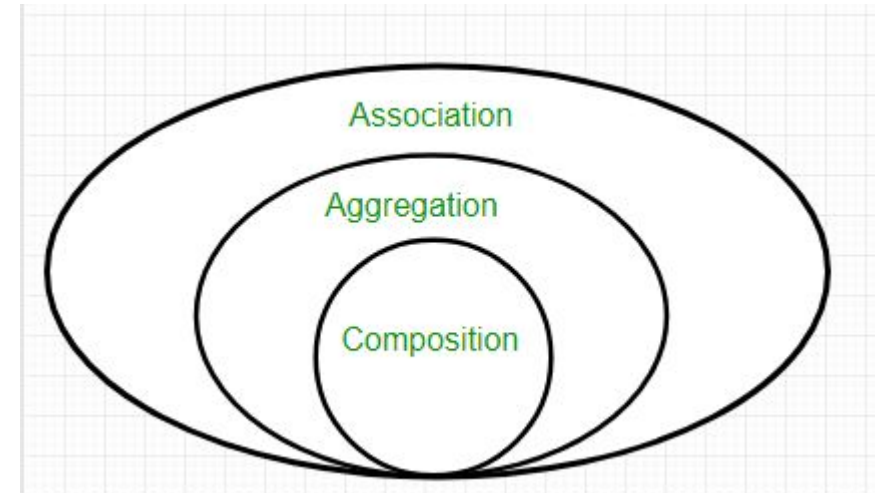


MR. MUHAMMAD SALEEM
LECTURER
DEPARTMENT OF COMPUTER SCIENCE

Dawood University of Engineering and Technology, Karachi

INTRODUCTION OF ASSOCIATION

- In object-oriented programming, relationships between classes play a crucial role in defining how objects interact with each other.
- Java, being an object-oriented language, provides mechanisms to model these relationships through association, aggregation, and composition.
- These describes how instances of classes relate to each other.



ASSOCIATION

- Association is the cardinal concept in object-oriented programming that describes the relationship between two independent classes.
- Association can be viewed as describing a "uses-a" relationship where an object uses, or in any way interacts with, another object.
- Association may be either unidirectional or bidirectional and may exist in several forms, such as one-to-one, one-to-many, many-to-one, and many-to-many.

Types of Association

```
graph TD; A[Types of Association] --> B[IS-A]; A --> C[HAS-A]; C --> D[Aggregation]; C --> E[Composition];
```

IS-A

HAS-A

Aggregation

Composition

TYPES OF ASSOCIATION

- **Unidirectional Association:** This association is the one in which one class is aware and associated with another class; the reverse is not true. For example, the Student class can be associated with the LibraryCard class, for the association where the student has a library card; a LibraryCard does not need to 'know about' a Student.
- **Bidirectional Association:** In this type of association, the classes are aware of each other and interact with one another. For example, a Teacher class and a Classroom class may be associated bidirectionally; there would be a teacher assigned to a classroom, and a classroom would know to which teacher it is assigned.

TYPES

```
class Department {
    String deptName;

    Department(String deptName) {
        this.deptName = deptName;
    }
}

class Employee {
    String empName;
    Department dept; // unidirectional association

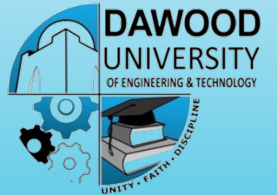
    Employee(String empName, Department dept) {
        this.empName = empName;
        this.dept = dept;
    }

    void display() {
        System.out.println(empName + " works in " + dept.deptName + " department");
    }
}

public class Test {
    public static void main(String[] args) {

        Department d = new Department("Computer Science");
        Employee e = new Employee("Sara", d);

        e.display();
    }
}
```



UML REPRESENTATION

unidirectional



bidirectional

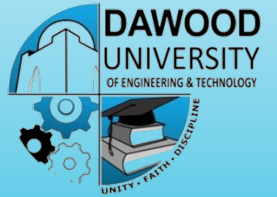


TYPES OF ASSOCIATION (BIDIRECTIONAL)

```
class Teacher {  
  
    String name;  
    Student student; // reference to Student  
  
    Teacher(String name) {  
        this.name = name;  
    }  
  
    void setStudent(Student s) {  
        student = s;  
    }  
  
    void showStudent() {  
        System.out.println(name + " teaches " + student.name);  
    }  
}
```

```
class Student {  
  
    String name;  
    Teacher teacher; // reference to Teacher  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void setTeacher(Teacher t) {  
        teacher = t;  
    }  
  
    void showTeacher() {  
        System.out.println(name + " is taught by " + teacher.name);  
    }  
}
```

ASSIGNMENT

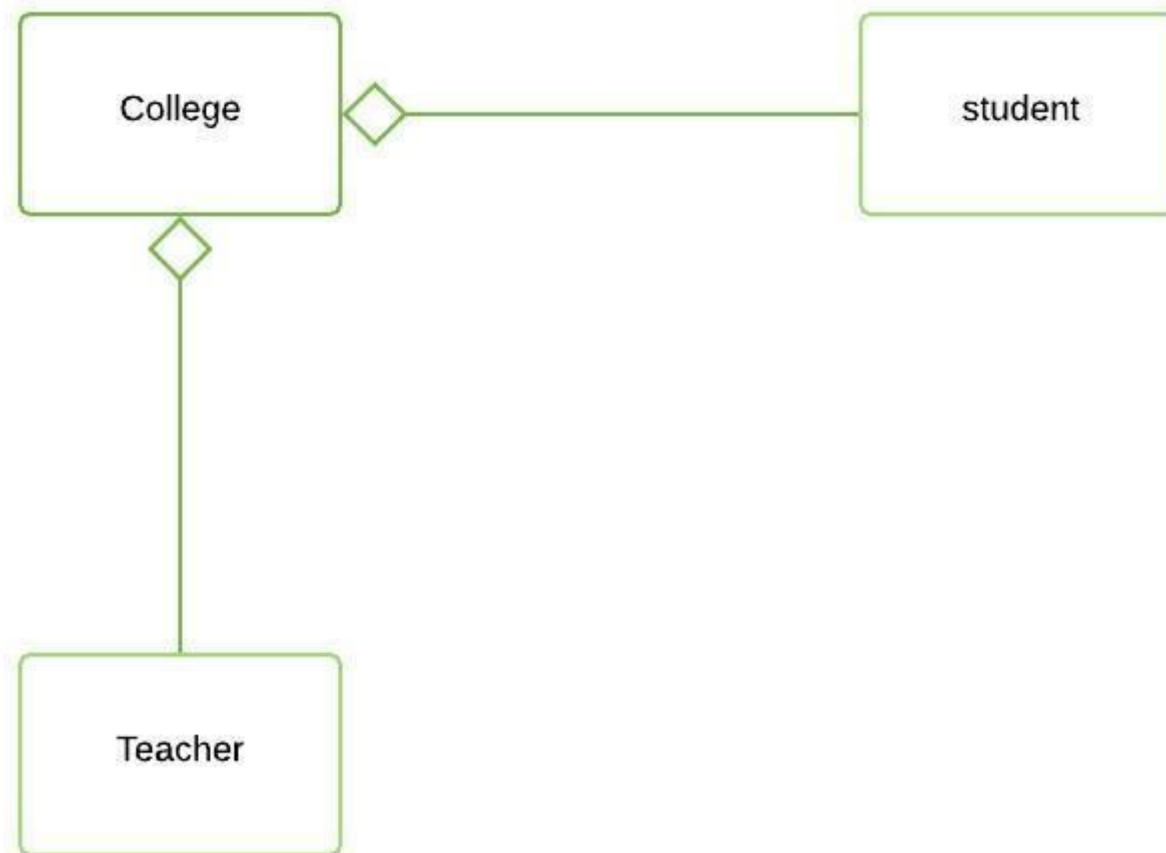


- Go and explore more about different types of relationships theoretically and practical implementations
 - 1 to 1
 - 1 to M
 - M to 1
 - M to M

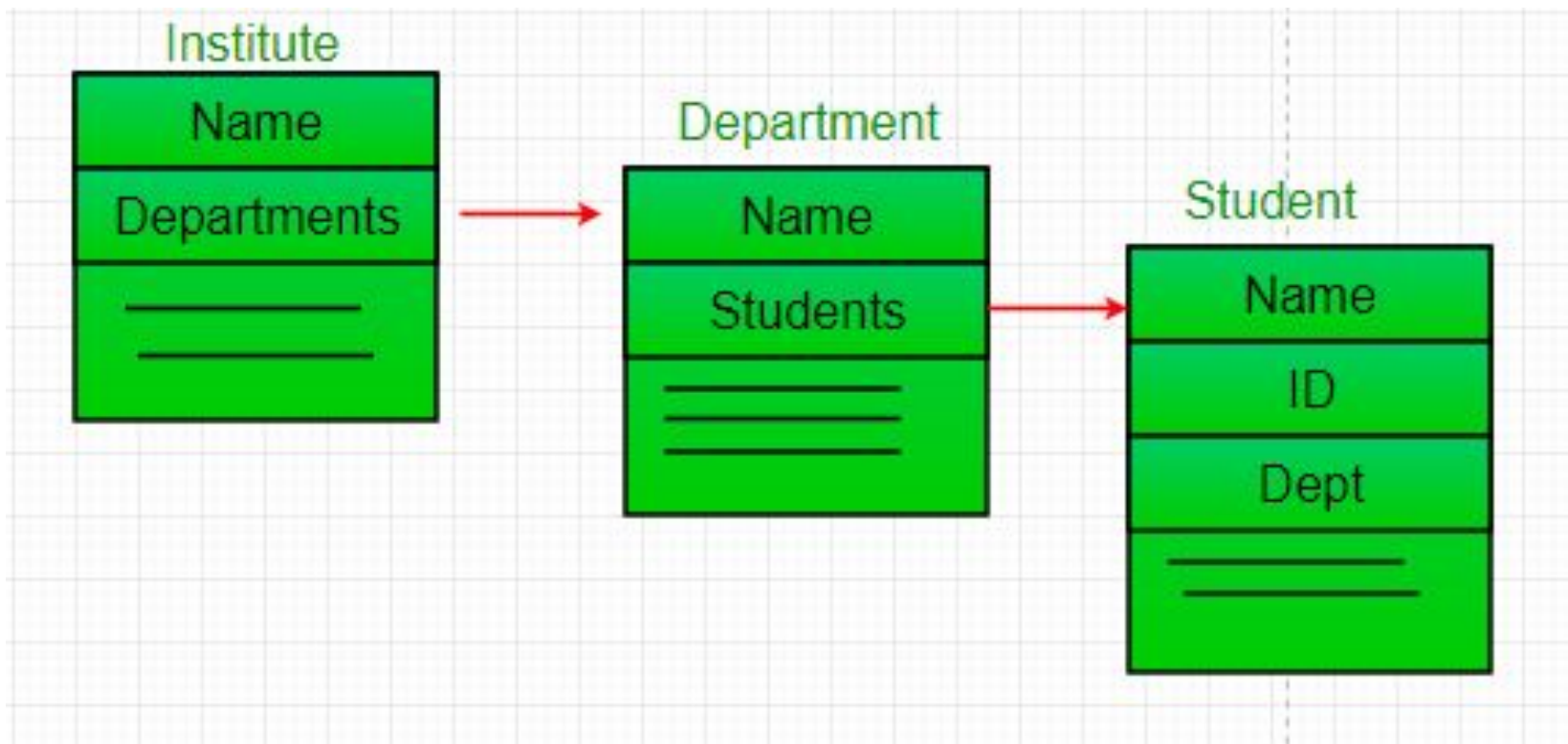
AGGREGATION

- Aggregation is an object-oriented programming relationship that represents a “has-a” association with weak ownership. In aggregation, the contained object can exist independently of the container, making it useful for building modular and reusable software components.
- **It is a special form of Association where:**
- It represents Has-A's relationship.
- It is a **unidirectional association** i.e. a one-way relationship Wallet and Money classes. Wallet has Money but money doesn't need to have Wallet necessarily so its a one directional relationship. but vice versa is not possible and thus unidirectional in nature.
- In Aggregation, **both entries can survive individually** which means ending one entity will not affect the other entity.

AGGREGATION



AGGREGATION



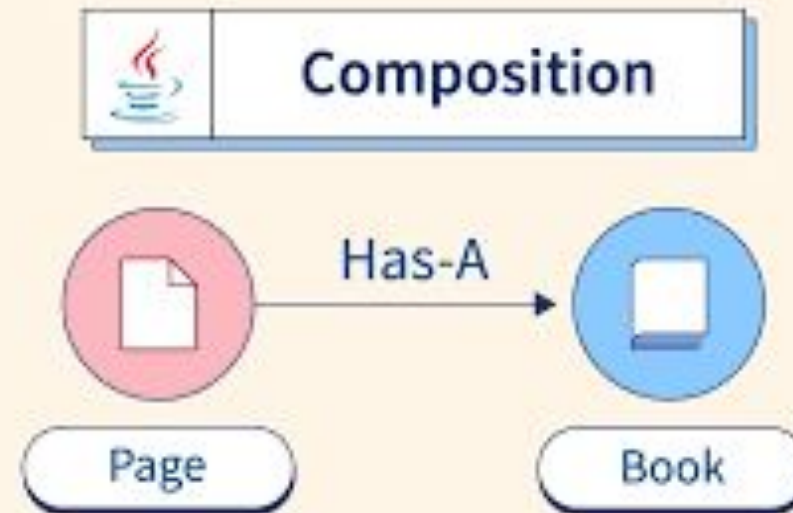
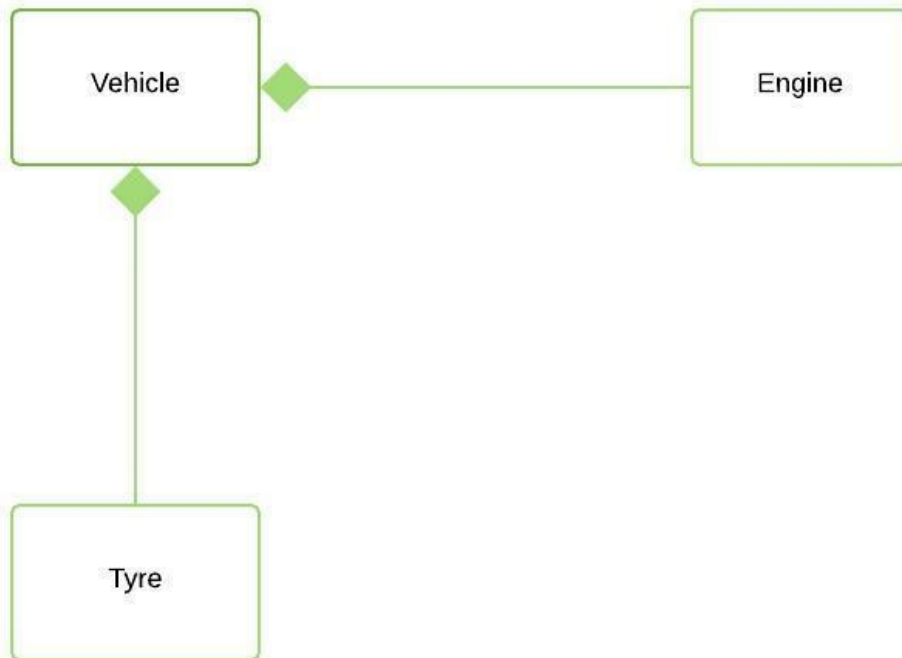
COMPOSITION

- It is a restricted form of Aggregation in which two entities (or you can say classes) are highly dependent on each other.
- For e.g. Human and Heart. A human needs heart to live and a heart needs a Human body to survive.
- In other words when the classes (entities) are dependent on each other and their life span are same (if one dies then another one too) then its a composition. Heart class has no sense if Human class is not present.

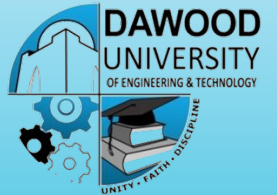
COMPOSITION

- Composition is a strong “part-of” relationship in object-oriented programming where the contained object’s lifecycle is completely dependent on the container object. It represents a restricted form of aggregation in which parts cannot exist independently of the whole, making it ideal for modeling tightly coupled components in complex systems.
- It represents **part-of** relationship.
- In composition, both entities are dependent on each other.
- When there is a composition between two entities, the composed object **cannot exist** without the other entity.

COMPOSITION UML



COMPOSITION



```
class Department {
    String name;

    // Constructor
    Department(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Department: " + name);
    }
}
```

```
class University {
    String uniName;
    Department csDept; // composition
    Department eeDept; // composition

    // University creates Departments inside it
    University(String uniName) {
        this.uniName = uniName;
        this.csDept = new Department("Computer Science");
        this.eeDept = new Department("Electrical Engineering");
    }

    void display() {
        System.out.println("University: " + uniName);
        csDept.display();
        eeDept.display();
    }
}
```

COMPOSITION



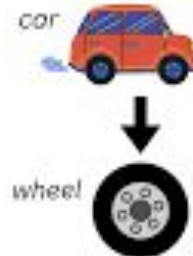
```
public class Main {  
    public static void main(String[] args) {  
        University du = new University("DUET, Karachi");  
  
        du.display();  
  
        // Cannot create Department independently here  
        // Department d = new Department("Math"); <-- optional, but not part of University  
    }  
}
```

ASSOCIATION VS AGGREGATION VS COMPOSITION

Association



Aggregation



Composition



ASSOCIATION VS AGGREGATION VS COMPOSITION

Feature	Association	Aggregation	Composition
Definition	General relationship between two classes	A special form of association with a "has-a" relationship	A stronger form of association with a "part-of" relationship
Dependency	Classes are related but can exist independently	Contained objects can exist independently of the container object	Contained objects cannot exist without the container object
Lifecycle	Independent lifecycles	Independent lifecycles	Dependent lifecycles
Ownership	No ownership implied	Shared ownership	Exclusive ownership
Strength	Weak	Moderate	Strong
Cardinality	One-to-one, one-to-many, many-to-one, many-to-many	One-to-one, one-to-many, many-to-one, many-to-many	One-to-one, one-to-many
Example	Teacher and Student	Library and Books	Car and Engine
Representation	Uses a direct reference	Uses a reference to the contained object(s)	Contains instances of the contained object(s)
Illustrative Example Code	<pre>java class Teacher { private Student student; }</pre>	<pre>java class Library { private List<Book> books; }</pre>	<pre>java class Car { private Engine engine; }</pre>

10	20	30	40	50	60	70	80	90	100
----	----	----	----	----	----	----	----	----	-----

newArr

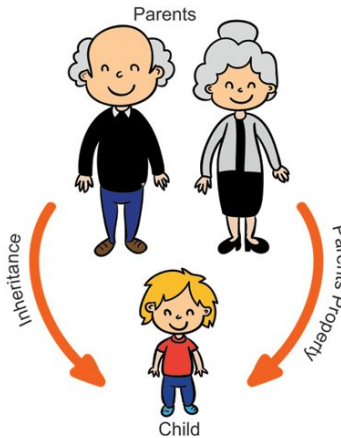
10	20	30	40	50	60	70	80	90	100										
----	----	----	----	----	----	----	----	----	-----	--	--	--	--	--	--	--	--	--	--

myArr

10	20	30	40	50	60	70	80	90	100										
----	----	----	----	----	----	----	----	----	-----	--	--	--	--	--	--	--	--	--	--



INHERITANCE



INHERITANCE

- **Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea behind inheritance in Java is that you can create new **classes** that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.
- Moreover, you can add new methods and fields in your current class also.

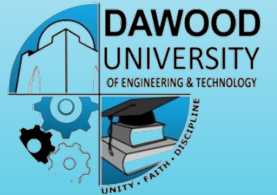
TERMINOLOGY USED IN INHERITANCE

- **Class:**
 - A class is a group of objects which have common properties.
 - It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:**
 - Subclass is a class which inherits the other class.
 - It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:**
 - Superclass is the class from where a subclass inherits the features.
 - It is also called a base class or a parent class.
- **Reusability:**
 - As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class.
 - You can use the same fields and methods already defined in the previous class.

EXTENDS KEYWORD

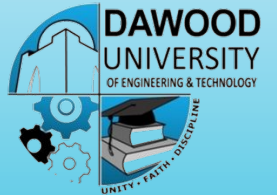
- The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.
- Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

EXAMPLE



```
class Employee{  
    float salary=40000;  
}  
  
class Programmer extends Employee{  
    int bonus=10000;  
  
    public static void main(String args[]){  
        Programmer p=new Programmer();  
        System.out.println("Programmer salary is:"+p.salary);  
        System.out.println("Bonus of Programmer is:"+p.bonus);  
    }  
}
```

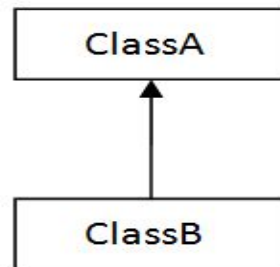
TYPES OF INHERITANCE



- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.
- In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

SINGLE INHERITANCE

- When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

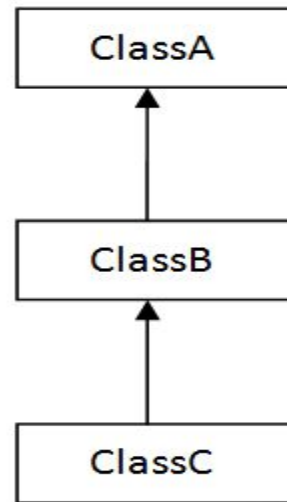


1) Single

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class TestInheritance{  
public static void main(String args[]){  
Dog d=new Dog();  
d.bark();  
d.eat();  
}}}
```

MULTILEVEL INHERITANCE EXAMPLE

- When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class. so there is a multilevel inheritance.

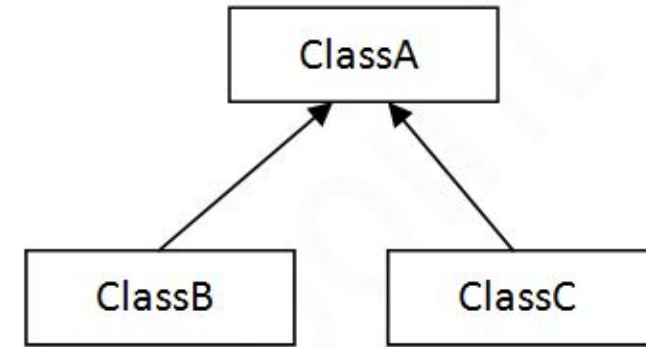


2) Multilevel

```
class Animal{
    void eat(){System.out.println("eating...")
    ;}
}
class Dog extends Animal{
    void bark(){System.out.println("barking.
    ..");}
}
class BabyDog extends Dog{
    void weep(){System.out.println("weepin
    g...");}
}
class TestInheritance2{
    public static void main(String args[]){
        BabyDog d=new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}
```

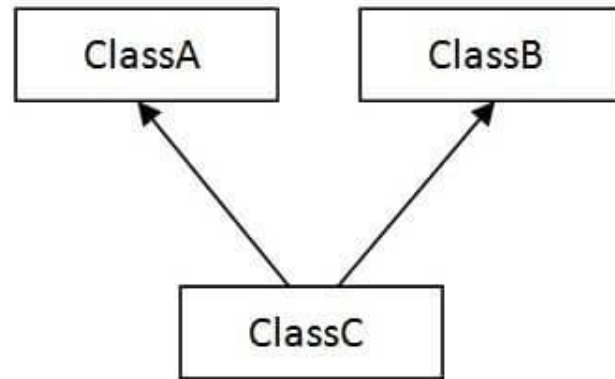
HIERARCHICAL INHERITANCE EXAMPLE

- When two or more classes inherit a single class, it is known as *hierarchical inheritance*. In the example given below, Dog and Cat classes inherit the Animal class, so there is hierarchical inheritance.

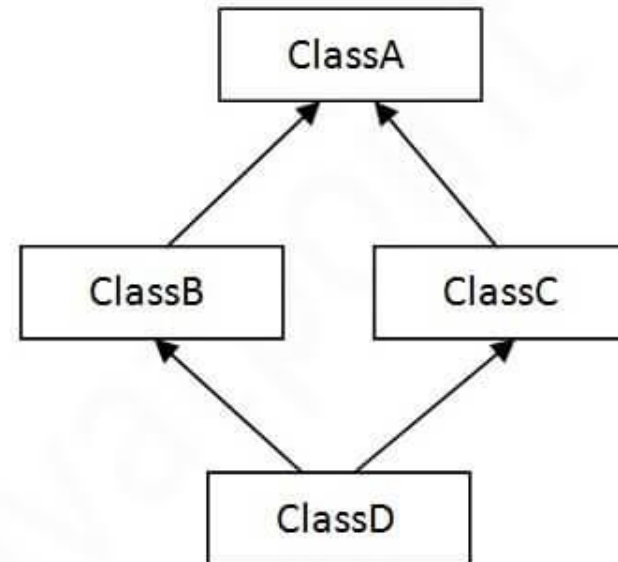


3) Hierarchical

MULTIPLE INHERITANCE AND HYBRID INHERITANCE



4) Multiple



5) Hybrid

Ambiguity

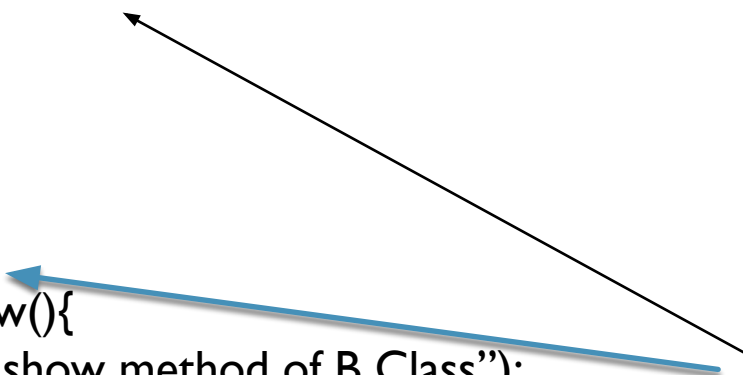
```
class A{  
    public void show(){  
        SoP("I am show method of A Class");  
    }  
}
```

```
class B{  
    public void show(){  
        SoP("I am show method of B Class");  
    }  
}
```

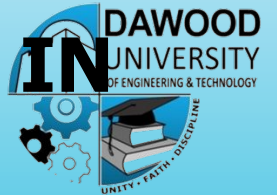
```
class C extends A,B{  
    psvm(String ar[]){
```

```
        C c = new C();  
        c.show();
```

```
    }  
}
```



Q) WHY MULTIPLE INHERITANCE IS NOT SUPPORTED IN JAVA?



- To reduce the complexity and simplify the language, multiple inheritance is not supported in java.
- Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.
- Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

MULTIPLE INHERITANCE

```
class A{  
void msg(){System.out.println("Hello");}  
}  
class B{  
void msg(){System.out.println("Welcome");}  
}  
class C extends A,B{//suppose if it were  
  
public static void main(String args[]){  
    C obj=new C();  
    obj.msg();//Now which msg() method would be invoked?  
}  
}
```



Thanks!

For The Time & Attention

Do You Have Any Question?

msaleem.jcb@gmail.com

+923333217216

<https://www.tpointtech.com/java-tutorial>